

The case for call-by-name

36th Symposium on Implementation and Application of Functional Languages

Edsko de Vries

August 26–28, 2024, Nijmegen, Netherlands



Problems

```
numberLines :: IO ()
numberLines =
  loop (enumFrom zero)
where
  loop :: [Int] -> IO ()
  loop (n:ns) = do
```

```

numberLines :: IO ()
numberLines =
    loop (enumFrom zero)
  where
    loop :: [Int] -> IO ()
    loop (n:ns) = do
        eof <- hIsEOF stdin
        if eof then return ()
            else do xs <- hGetLine stdin
                    numberLine n xs
                    loop ns

numberLine :: Int -> String -> IO ()
numberLine n xs = putStrLn $ show n ++ ": " ++ xs

```

```
numberLines :: IO ()
numberLines =
    loop (enumFrom zero)
  where
    loop :: [Int] -> IO ()
    loop (n:ns) = do
        eof <- hIsEOF stdin
        if eof then return ()
            else do xs <- hGetLine stdin
                    numberLine n xs
                    loop ns

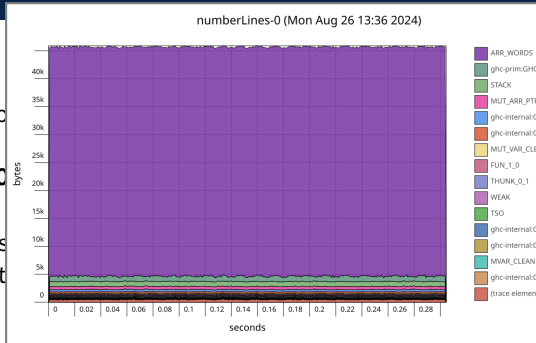
numberLine :: Int -> String -> IO ()
numberLine n xs = putStrLn $ show n ++ ": " ++ xs

main :: IO ()
main = numberLines
```

```

numberLines :: IO ()
numberLines =
    loop (enumFrom zero)
  where
    loop :: [Int] -> IO ()
    loop (n:ns) = do
        eof <- hIsEOF s
        if eof then ret
        else do

```



```

    loop ns

```

```

numberLine :: Int -> String -> IO ()
numberLine n xs = putStrLn $ show n ++ ": " ++ xs

main :: IO ()
main = numberLines

```

```
numberLines :: IO ()
numberLines =
    loop (enumFrom zero)
  where
    loop :: [Int] -> IO ()
    loop (n:ns) = ..

main :: IO ()
main = tryAgainOn UserInterrupt $ numberLines
```

```
numberLines :: IO ()
numberLines =
    loop (enumFrom zero)
  where
    loop :: [Int] -> IO ()
    loop (n:ns) = ..

main :: IO ()
main = tryAgainOn UserInterrupt $ numberLines

tryAgainOn :: (Exception e, Eq e) => e -> IO a -> IO a
tryAgainOn e io =
    io `catch` \e' ->
        if e == e' then io
        else throwIO e'
```



```

numberLines :: IO ()
numberLines =
    loop (enumFrom zero)
  where
    loop :: [Int] -> IO ()
    loop (n:ns) = ..

```

```

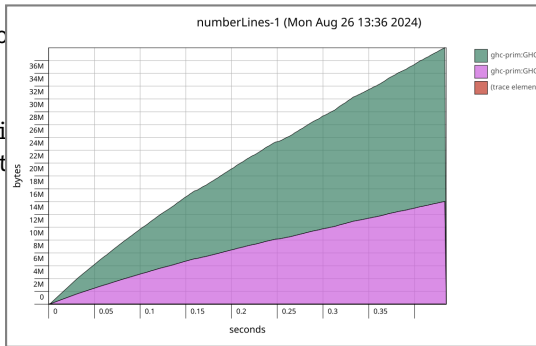
main :: IO ()
main = tryAgainOn UserInterrupt $ numberLines

```

```

tryAgainOn :: (Exception -> IO ()) -> IO ()
tryAgainOn e io =
    io `catch` \e' ->
        if e == e' then i
        else t

```



Problems

**Not all heap
is data**

Conduit

```
data Conduit i o m r =  
  Done r  
| Yield o (Conduit i o m r)  
| Await (Maybe i -> Conduit i o m r)  
| Lift (m (Conduit i o m r))
```

Conduit

```
data Conduit i o m r =  
    Done r  
  | Yield o (Conduit i o m r)  
  | Await (Maybe i -> Conduit i o m r)  
  | Lift (m (Conduit i o m r))  
  
(.|) :: (..) => Conduit a b m ()  
        -> Conduit b c m r  
        -> Conduit a c m r
```

Conduit

```
data Conduit i o m r =  
    Done r  
  | Yield o (Conduit i o m r)  
  | Await (Maybe i -> Conduit i o m r)  
  | Lift (m (Conduit i o m r))  
  
(.|) :: (..) => Conduit a b m ()  
      -> Conduit b c m r  
      -> Conduit a c m r  
  
run  :: (..) => Conduit Void Void m r -> m r
```

Conduit

```
data Conduit i o m r =  
    Done r  
  | Yield o (Conduit i o m r)  
  | Await (Maybe i -> Conduit i o m r)  
  | Lift (m (Conduit i o m r))  
  
(.|) :: (..) => Conduit a b m ()  
      -> Conduit b c m r  
      -> Conduit a c m r  
  
run  :: (..) => Conduit Void Void m r -> m r  
  
hGetChars :: Handle -> Conduit Void Char IO ()  
hGetChars h = go  
  where  
    go = Lift $ do  
      eof <- hIsEOF h  
      if eof then return $ Done ()  
      else do c <- hGetChar h  
              return $ Yield c go
```

```
count :: Conduit a Void m Int
count = loop 0
  where
    loop :: Int -> Conduit a Void m Int
    loop !acc =
      Await $ \x -> case x of
        Nothing -> Done acc
        Just _   -> loop (acc + 1)
```

```
count :: Conduit a Void m Int
count = loop 0
  where
    loop :: Int -> Conduit a Void m Int
    loop !acc =
      Await $ \x -> case x of
        Nothing -> Done acc
        Just _   -> loop (acc + 1)

main :: IO ()
main =
  tryAgainOn UserInterrupt $
    withFile "lotsalines" ReadMode $ \h -> do
      let conduit = hGetChars h .| count
      print =<< run conduit
```



```

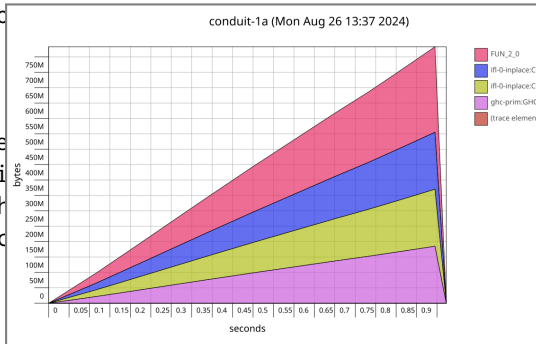
count :: Conduit a Void m Int
count = loop 0
  where
    loop :: Int -> Conduit a Void m Int
    loop !acc =
      Await $ \x -> case x of
        Nothing -> Done acc
        Just _ -> ld

```

```

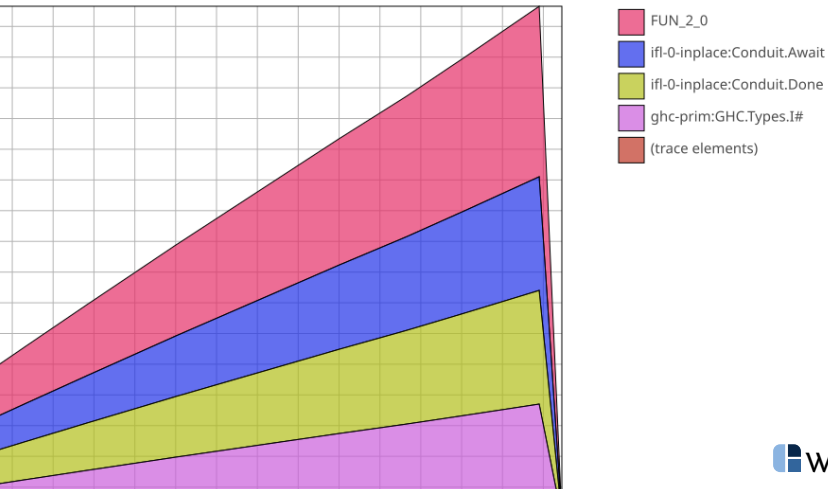
main :: IO ()
main =
  tryAgainOn UserInte
  withFile "lotsali
    let conduit = h
  print =<< run c

```



```
count :: Conduit a Void m Int
count = loop 0
```

conduit-1a (Mon Aug 26 13:37 2024)



Problems

Intermediate data

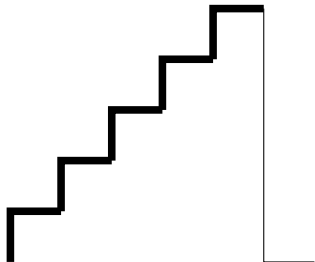
```
process :: [Int] -> IO ()
process [] = return ()
process xs = do
    xs' <- evaluate $ drop 5 xs
    performMinorGC
    process xs'
```

```
demo :: IO ()
demo = do
    let left  = [100 .. 104]
        right = [200 .. 204]

    process left
    process right
```

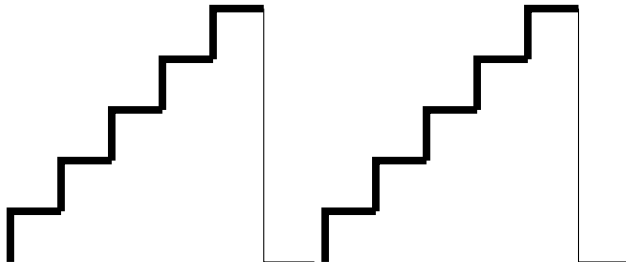
(Idealized) heap

```
let left  = [100 .. 104]  
    right = [200 .. 204]  
process left  
process right
```



(Idealized) heap

```
let left  = [100 .. 104]  
    right = [200 .. 204]  
process left  
process right
```



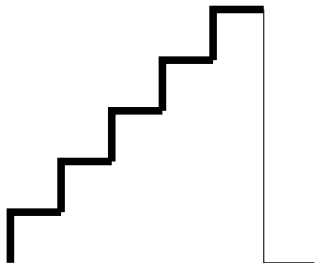
```
process :: [Int] -> IO ()
process [] = return ()
process xs = do
    xs' <- evaluate $ drop 5 xs
    performMinorGC
    process xs'
```

```
demo :: IO ()
demo = do
    let left  = [100 .. 119]
        right = [200 .. 219]

    process left
    process right
```

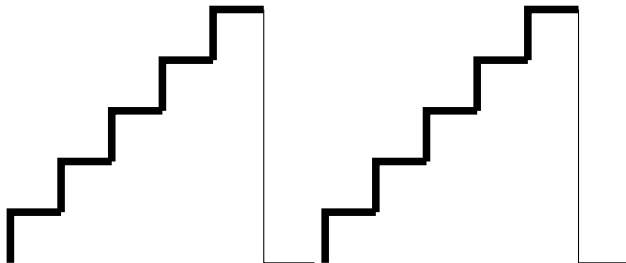
(Idealized) heap

```
let left  = [100 .. 119]  
    right = [200 .. 219]  
process left  
process right
```



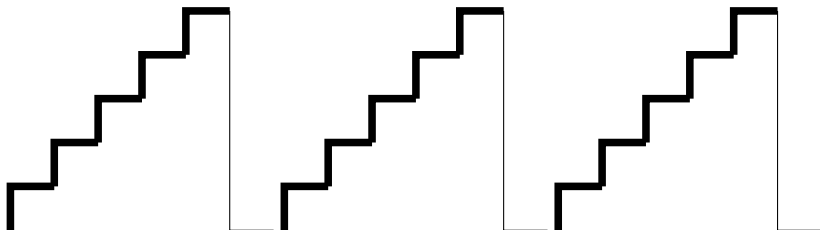
(Idealized) heap

```
let left  = [100 .. 119]  
    right = [200 .. 219]  
process left  
process right
```



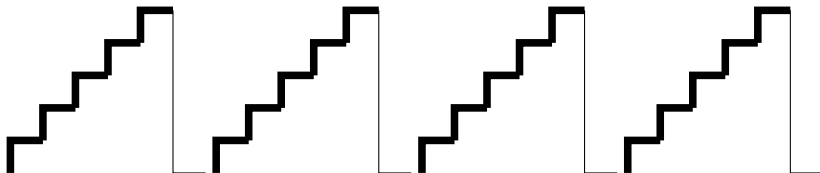
(Idealized) heap

```
let left  = [100 .. 119]  
    right = [200 .. 219]  
process left  
process right
```



(Idealized) heap

```
let left  = [100 .. 119]  
    right = [200 .. 219]  
process left  
process right
```



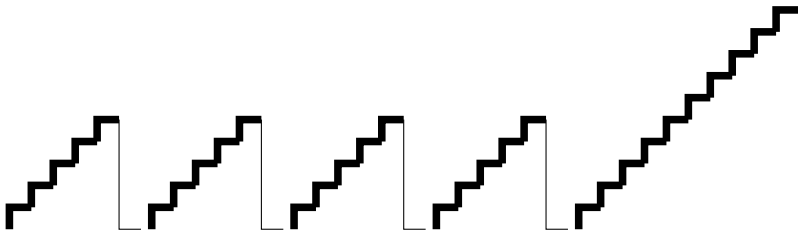
(Idealized) heap

```
let left  = [100 .. 119]  
    right = [200 .. 219]  
process left  
process right
```



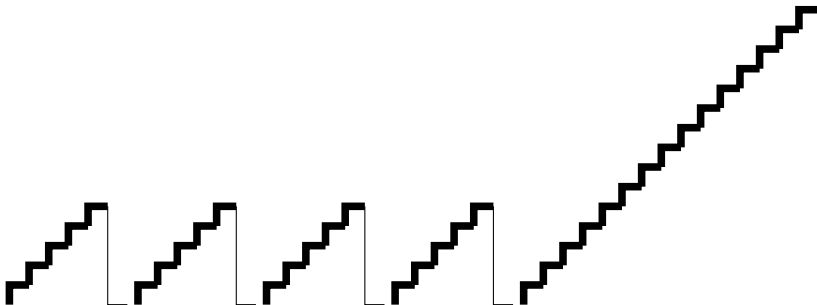
(Idealized) heap

```
let left  = [100 .. 119]  
    right = [200 .. 219]  
process left  
process right
```



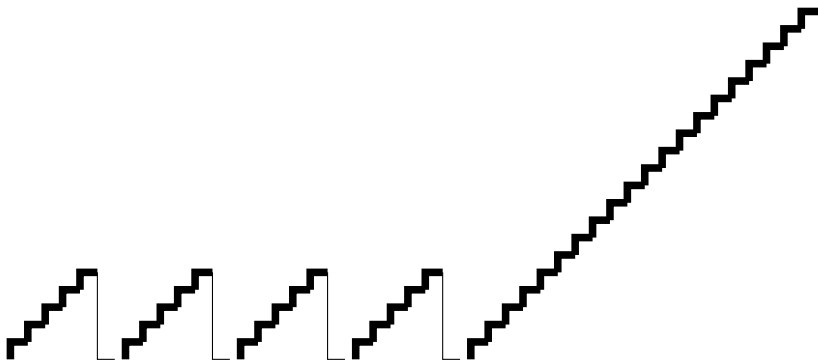
(Idealized) heap

```
let left  = [100 .. 119]  
    right = [200 .. 219]  
process left  
process right
```



(Idealized) heap

```
let left  = [100 .. 119]  
    right = [200 .. 219]  
process left  
process right
```



Analysis


```
numberLines :: IO ()
numberLines =
    loop (enumFrom zero)
  where
    loop :: [Int] -> IO ()
    loop (n:ns) = ..

main :: IO ()
main = tryAgainOn UserInterrupt $ numberLines
```

```
count :: Conduit a Void m Int
count = loop 0
  where
    loop :: Int -> Conduit a Void m Int
    loop !acc =
      Await $ \x -> case x of
        Nothing -> Done acc
        Just _   -> loop (acc + 1)

main :: IO ()
main =
  tryAgainOn UserInterrupt $
    withFile "lotsalines" ReadMode $ \h -> do
      let conduit = hGetChars h .| count
      print =<< run conduit
```

```
count :: Conduit a Void m Int
count = loop 0
  where
    loop :: Int -> Conduit a Void m Int
    loop !acc =
      let recurse = loop (acc + 1) in
      Await $ \x -> case x of
        Nothing -> Done acc
        Just _   -> recurse

main :: IO ()
main =
  tryAgainOn UserInterrupt $
    withFile "lotsalines" ReadMode $ \h -> do
      let conduit = hGetChars h .| count
      print =<< run conduit
```

```

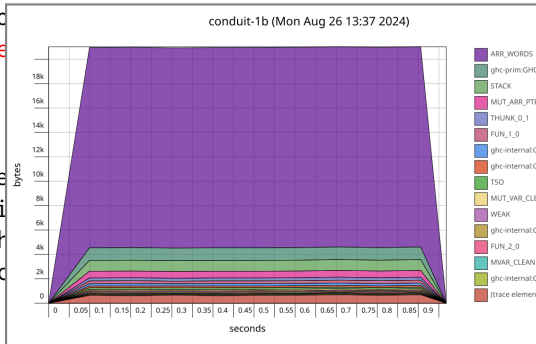
count :: Conduit a Void m Int
count = loop 0
  where
    loop :: Int -> Conduit a Void m Int
    loop !acc =
      let recurse = loop (acc + 1) in
      Await $ \x -> case x of
        Nothing -> Do
        Just _ -> re

```

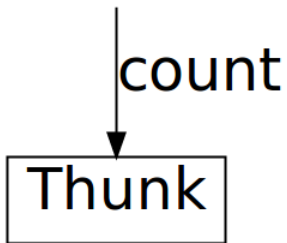
```

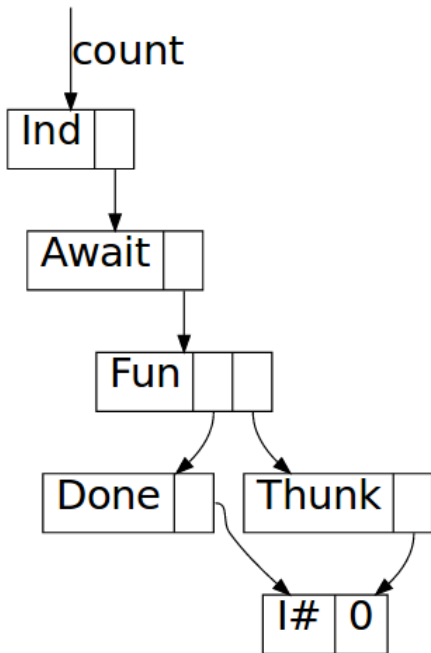
main :: IO ()
main =
  tryAgainOn UserInte
    withFile "lotsali
      let conduit = h
      print =<< run c

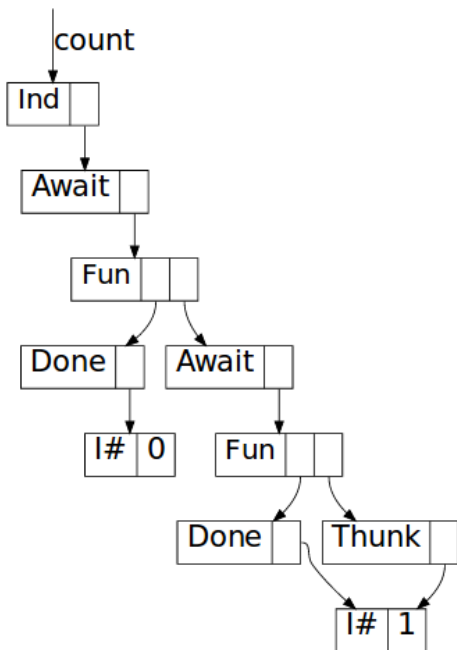
```



(Full laziness disabled)







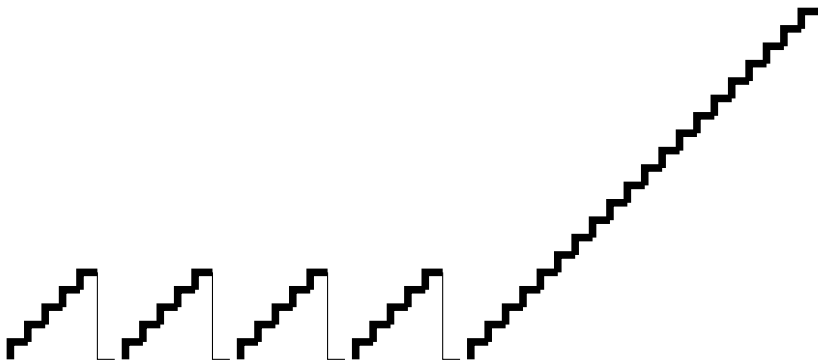
```
process :: [Int] -> IO ()
process [] = return ()
process xs = do
    xs' <- evaluate $ drop 5 xs
    performMinorGC
    process xs'
```

```
demo :: IO ()
demo = do
    let left  = [100 .. 104]
        right = [200 .. 204]

    process left
    process right
```


(Idealized) heap

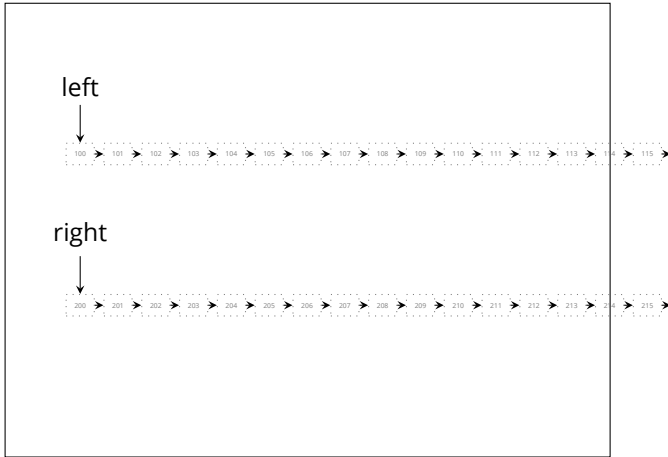
```
let left  = [100 .. 119]  
    right = [200 .. 219]  
process left  
process right
```



Gen 1



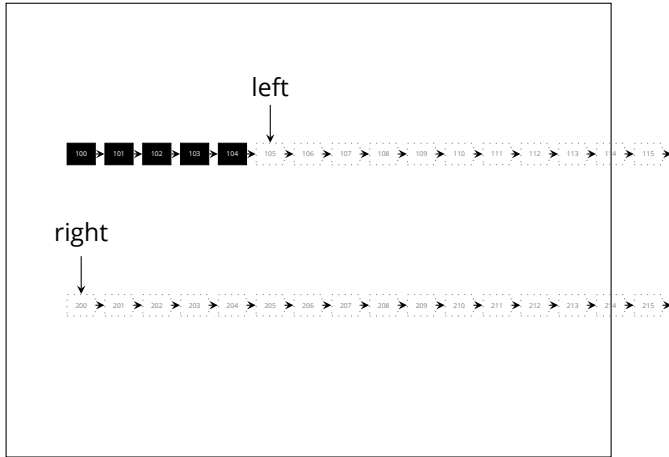
Gen 0



Gen 1



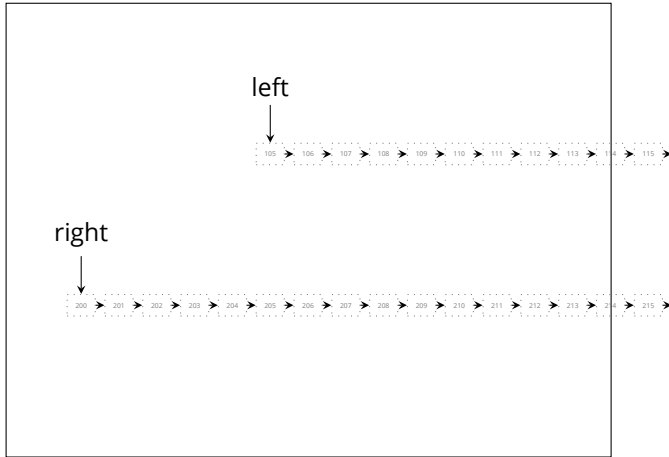
Gen 0



Gen 1



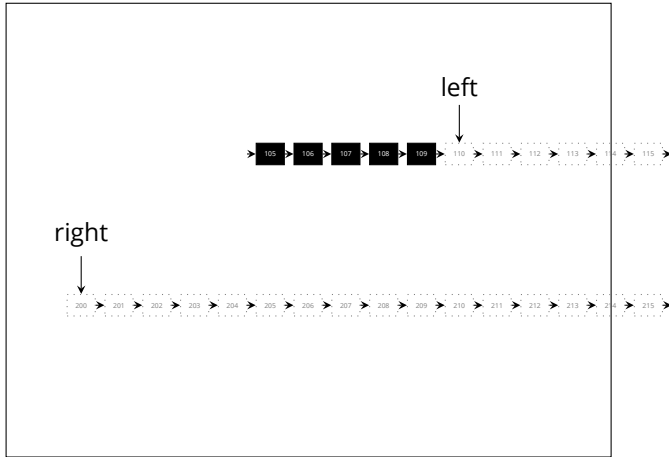
Gen 0



Gen 1



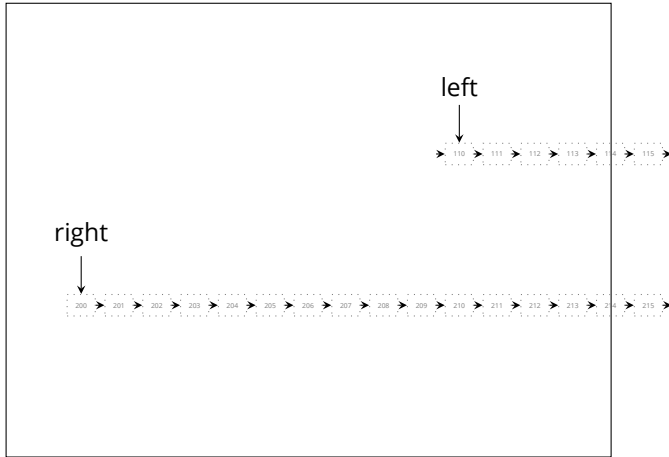
Gen 0



Gen 1



Gen 0

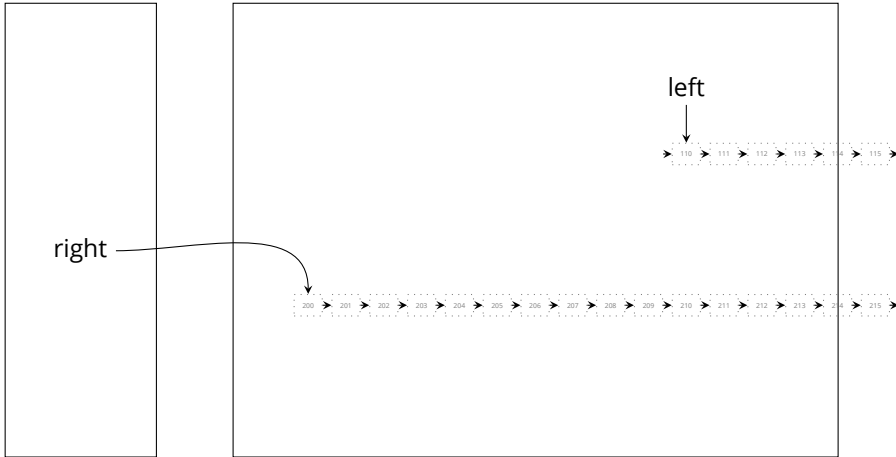


Gen 1

Gen 0

right

left



Towards a solution

Preventing sharing


```
numberLines :: IO ()
numberLines =
    loop (enumFrom zero)
where
    loop :: [Int] -> IO ()
    loop (n:ns) = ..
```

```
numberLines :: IO ()
numberLines = do
    let ns = enumFrom zero
    loop ns
  where
    loop :: [Int] -> IO ()
    loop (n:ns) = ..
```

The IO monad

```
newtype IO a = IO {  
    unIO ::  
        State# RealWorld  
    -> (# State# RealWorld, a #)  
}
```

(module GHC.Types in package ghc-prim)

```
numberLines :: IO ()  
numberLines = do  
    let ns = enumFrom zero  
    loop ns
```

```
numberLines :: IO ()  
numberLines = do  
    let ns = enumFrom zero  
    loop ns
```

```
numberLines :: IO ()  
numberLines =  
  
    IO $ \w0 ->  
        let ns = enumFrom zero in  
  
        loop ns w0
```

```
numberLines :: IO ()
numberLines = do
    let ns = enumFrom zero
    loop ns
```

```
numberLines :: IO ()
numberLines =
```

```
IO $ \w0 ->
    let ns = enumFrom zero in

    loop ns w0
```

```
numberLines :: IO ()
numberLines =
```

```
let ns = enumFrom zero in
IO $ \w0 ->

    loop ns w0
```

Changes from Haskell 1.2 to Haskell 1.3 (May 1996)

(..)

The Monad class is now an important part of the Haskell language. (..) The do syntax has been added to Haskell to provide a convenient syntax for monadic binding:

Syntax:

`exp -> do {stmt}`

`stmt -> exp`

`stmt -> exp ; stmt`

`stmt -> let decls ; stmt`

`stmt -> var <- exp ; stmt`

Expansion:

`exp`

`exp >> do stmt`

`let decls in do stmt`

`exp >>= \var -> do stmt`

```
numberLines :: IO ()
numberLines = do
  let ns = enumFrom zero
  loop ns
```

```
numberLines :: IO ()
numberLines =
```

```
IO $ \w0 ->
  let ns = enumFrom zero in

  loop ns w0
```

```
numberLines :: IO ()
numberLines =
```

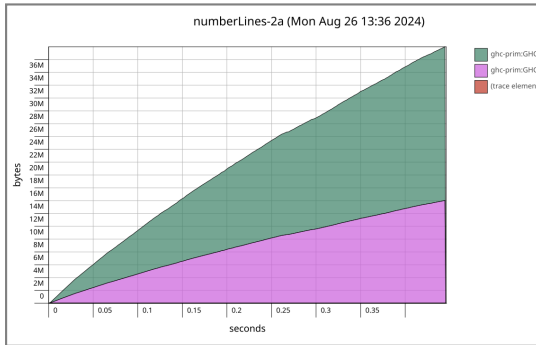
```
let ns = enumFrom zero in
IO $ \w0 ->

  loop ns w0
```



```
numberLines :: IO ()
numberLines = IO $ \w ->
    let ns = enumFrom zero
    unIO (loop ns) w
where
    loop :: [Int] -> IO ()
    loop (n:ns) = ..
```

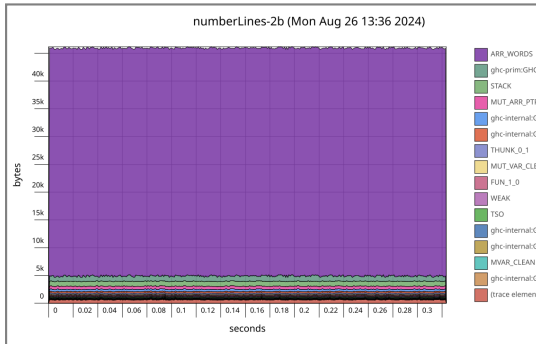
```
numberLines :: IO ()
numberLines = IO $ \w ->
    let ns = enumFrom zero
    unIO (loop ns) w
where
    loop :: [Int] -> IO ()
    loop (n:ns) = ..
```



```

numberLines :: IO ()
numberLines = IO $ \w ->
    let ns = enumFrom zero
    unIO (loop ns) w
where
    loop :: [Int] -> IO ()
    loop (n:ns) = ..

```



(Full laziness disabled)

```
numberLines :: IO ()
numberLines = do
    let ns = enumFrom zero
    noop
    loop ns
  where
    loop :: [Int] -> IO ()
    loop (n:ns) = ..
```

```
noop :: IO ()
noop = return ()
```

```

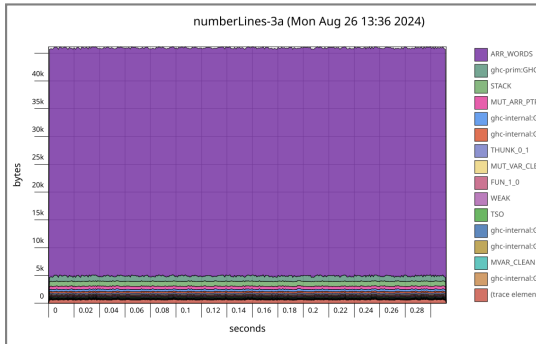
numberLines :: IO ()
numberLines = do
    let ns = enumFrom zero
    noop
    loop ns
  where
    loop :: [Int] -> IO ()
    loop (n:ns) = ..

```

```

noop :: IO ()
noop = return ()

```



```

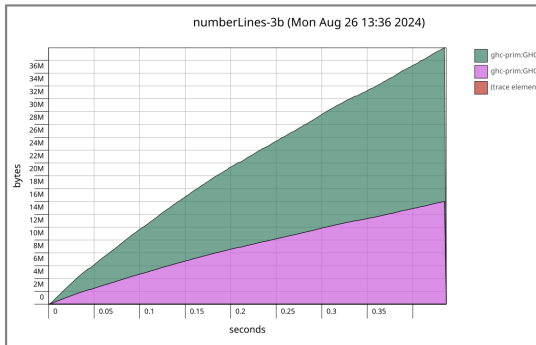
numberLines :: IO ()
numberLines = do
    let ns = enumFrom zero
    noop
    loop ns
  where
    loop :: [Int] -> IO ()
    loop (n:ns) = ..

```

```

noop :: IO ()
noop = return ()

```



(Full laziness enabled)

let .. ←
λx · ...
let ..

Full laziness

let ..
λx · ...
→ let ..

State hack

(x :: State# RealWorld)

Towards a solution

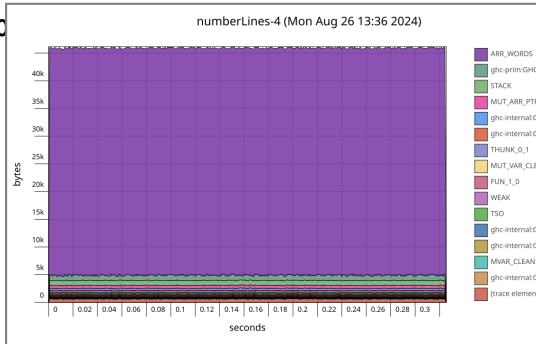
Preventing updates


```
numberLines :: IO ()
numberLines = do
  ns <- dupIO $! enumFrom zero
  loop ns
where
  loop :: [Int] -> IO ()
  loop (n:ns) = ..
```

```

numberLines :: IO ()
numberLines = do
  ns <- dupIO $! enumFrom zero
  loop ns
where
  loop :: [Int] -> IO ()
  loop (n:ns) = ..

```



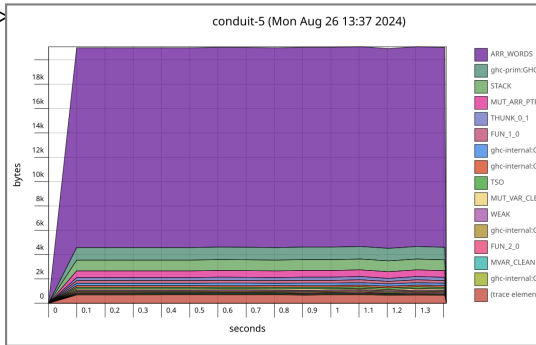
(Fragility in this example comes from the fact that we cannot duplicate CAFs.)

```
count :: Conduit a Void IO Int
count = loop 0
  where
    loop :: Int -> Conduit a Void IO Int
    loop !acc = Lift $ do
      recurse <- dupIO $ loop (acc + 1)
      return $
        Await $ \x -> case x of
          Nothing -> Done acc
          Just _   -> recurse
```

```

count :: Conduit a Void IO Int
count = loop 0
  where
    loop :: Int -> Conduit a Void IO Int
    loop !acc = Lift $ do
      recurse <- dupIO $ loop (acc + 1)
      return $
        Await $ \x ->
          Nothing ->
          Just _ ->

```



```
process :: [Int] -> IO ()
process [] = return ()
process xs = do
    xs' <- evaluate $ drop 5 xs
    performMinorGC
    process xs'
```

```
demo :: IO ()
demo = do
    let left  = [100 .. 119]
        right = [200 .. 219]

    process =<< dupIO left
    process =<< dupIO right
```

Gen 1

Gen 0

right

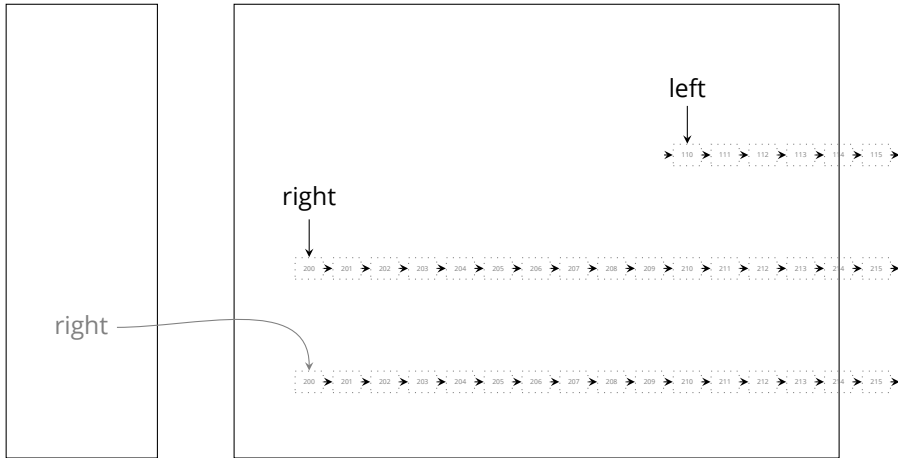
left

110 111 112 113 114 115

200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215

Gen 1

Gen 0



Limitations

Gen 1

Gen 0

right

left



Gen 1

Gen 0

right

right

left

110 111 112 113 114 115

200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215

```
count :: Conduit a Void m Int
count = loop 0
  where
    loop :: Int -> Conduit a Void m Int
    loop !acc =
      Await $ \x -> case x of
        Nothing -> Done acc
        Just _   -> loop (acc + 1)
```

```
count :: Conduit a Void m Int
count = loop 0
  where
    loop :: Int -> Conduit a Void m Int
    loop !acc =
      let recurse = loop (acc + 1) in
      Await $ \x -> case x of
        Nothing -> Done acc
        Just _   -> recurse
```

```
count :: Conduit a Void IO Int
count = loop 0
  where
    loop :: Int -> Conduit a Void IO Int
    loop !acc = Lift $ do
      recurse <- dupIO $ loop (acc + 1)
      return $ Await $ \x -> case x of
        Nothing -> Done acc
        Just _   -> recurse
```

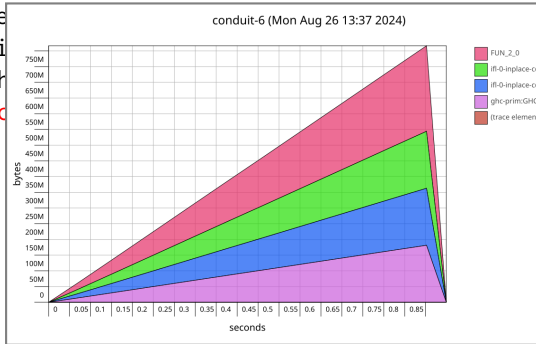
```
run :: Conduit Void Void IO r -> IO r
run = dupIO >=> \x -> case x of
    Done r -> return r
    Lift k -> k >=> run
```

```
run :: Conduit Void Void IO r -> IO r
run = dupIO >=> \x -> case x of
    Done r -> return r
    Lift k -> k >>= run

main :: IO ()
main =
    tryAgainOn UserInterrupt $
        withFile "lotsalines" ReadMode $ \h -> do
            let conduit = hGetChars h .| count
            print =<< run conduit
```

```
run :: Conduit Void Void IO r -> IO r
run = dupIO >=> \x -> case x of
    Done r -> return r
    Lift k -> k >=> run
```

```
main :: IO ()
main =
    tryAgainOn UserInte
    withFile "lotsali
    let conduit = h
    print =<< run c
```




```
count :: Conduit a Void m Int
count = loop 0
  where
    loop :: Int -> Conduit a Void m Int
    loop !acc =
      let recurse = loop (acc + 1) in
      Await $ \x -> case x of
        Nothing -> Done acc
        Just _   -> recurse
```

```
data Box a = Box a
```

```
data Conduit i o m r =  
    Done r  
  | Await (Maybe i -> Box (Conduit i o m r))  
  ..
```

```
data Box a = Box a
```

```
data Conduit i o m r =  
    Done r  
  | Await (Maybe i -> Box (Conduit i o m r))  
  ..
```

```
count :: Conduit a Void m Int
```

```
count = loop 0
```

```
  where
```

```
    loop :: Int -> Conduit a Void m Int
```

```
    loop !acc =
```

```
      let recurse = loop (acc + 1) in
```

```
      Await $ \x -> case x of
```

```
        Nothing -> Box $ Done acc
```

```
        Just _   -> Box $ recurse
```

```
data Box a = Box a
```

```
data Conduit i o m r =  
    Done r  
  | Await (Maybe i -> Box (Conduit i o m r))  
  ..
```

```
count :: Conduit a Void m Int
```

```
count = loop 0
```

```
  where
```

```
    loop :: Int -> Conduit a Void m Int
```

```
    loop !acc =
```

```
      Await $ \x -> case x of
```

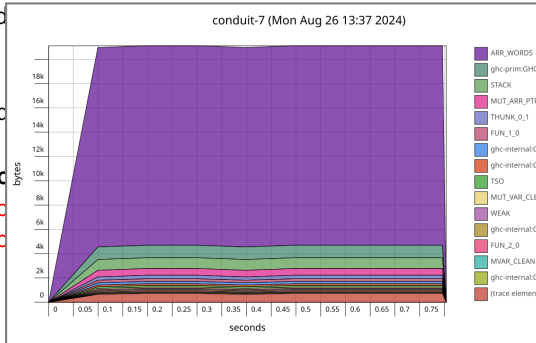
```
        Nothing -> Box $ Done acc
```

```
        Just _   -> Box $ loop (acc + 1)
```

```
data Box a = Box a
```

```
data Conduit i o m r =  
    Done r  
  | Await (Maybe i -> Box (Conduit i o m r))  
  ..
```

```
count :: Conduit a Void  
count = loop 0  
  where  
    loop :: Int -> Conduit a Void  
    loop !acc =  
      Await $ \x -> do  
        Nothing -> Box (Conduit i o m r)  
        Just _ -> Box (Conduit i o m r)
```



(Full laziness enabled)

```
await :: (Maybe i -> Conduit i o m r) -> ..  
await f = Await (\x -> Box (f x))
```

```
await :: (Maybe i -> Conduit i o m r) -> ..  
await f = Await (\x -> Box (f x))
```

```
count :: Conduit a Void m Int  
count = loop 0
```

where

```
loop :: Int -> Conduit a Void m Int
```

```
loop !acc =
```

```
    await $ \x -> case x of
```

```
        Nothing -> Done acc
```

```
        Just _   -> loop (acc + 1)
```

```
await :: (Maybe i -> Conduit i o m r) -> ..
await f = Await (\x -> Box (f x))
```

```
count :: Conduit a Void m Int
count = loop 0
```

where

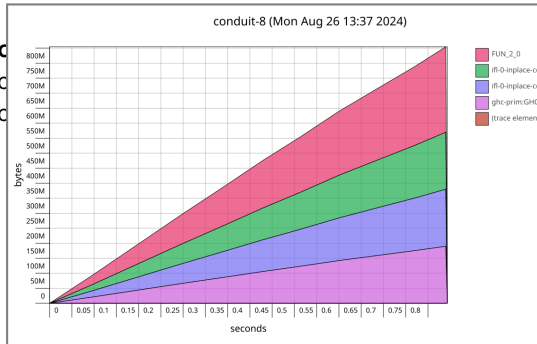
```
loop :: Int -> Conduit a Void m Int
```

```
loop !acc =
```

```
  await $ \x -> do
```

```
    Nothing -> do
```

```
    Just _ -> loop
```



(Full laziness enabled)

Language support

Preventing sharing

- ▶ “Full laziness (or let floating in general) considered harmful”?
Takes away our ability to control sharing.

Preventing sharing

- ▶ “Full laziness (or let floating in general) considered harmful”?
Takes away our ability to control sharing.
- ▶ Predictability versus performance

Preventing sharing

- ▶ “Full laziness (or let floating in general) considered harmful”?
Takes away our ability to control sharing.
- ▶ Predictability versus performance
- ▶ Difficult-to-predict consequences for compiler design

Preventing sharing

- ▶ “Full laziness (or let floating in general) considered harmful”?
Takes away our ability to control sharing.
- ▶ Predictability versus performance
- ▶ Difficult-to-predict consequences for compiler design
- ▶ Compositionally can be challenging

Embedding call-by-name: types

```
data {-# NOUPDATE #-} Conduit = ...
```

Embedding call-by-name: types

```
data {-# NOUPDATE #-} Conduit = ...
```

- ▶ What to do with polymorphic bindings?

Embedding call-by-name: types

```
data {-# NOUPDATE #-} Conduit = ...
```

- ▶ What to do with polymorphic bindings?
- ▶ Maybe monomorphic bindings are enough (no need to prevent **all** updates, we just need to prevent long chains)

Embedding call-by-name: kinds

```
data Conduit :: NotUpdatableType = ...
```

¹“Fixing a space leak by copying thunks”, Philipp Schuster

Embedding call-by-name: kinds

```
data Conduit :: NotUpdatableType = ...
```

- ▶ GHC already has a lifted kind (call-by-need) and an unlifted kind (call-by-value)¹

¹“Fixing a space leak by copying thunks”, Philipp Schuster

Embedding call-by-name: kinds

```
data Conduit :: NotUpdatableType = ...
```

- ▶ GHC already has a lifted kind (call-by-need) and an unlifted kind (call-by-value)¹
- ▶ Problem: ecosystem split; many general purpose functions would not be usable with these other-kinded types.

¹“Fixing a space leak by copying thunks”, Philipp Schuster

Embedding call-by-name: values

```
data Conduit =  
    {-# NOUPDATE #-} Yield ..  
  | {-# NOUPDATE #-} Await ..  
  ..
```

Embedding call-by-name: values

```
data Conduit =  
    {-# NOUPDATE #-} Yield ..  
  | {-# NOUPDATE #-} Await ..  
  ..
```

- Performance impact (dynamic rather than static check)

Embedding call-by-name: values

```
data Conduit =  
    {-# NOUPDATE #-} Yield ..  
  | {-# NOUPDATE #-} Await ..  
  ..
```

- ▶ Performance impact (dynamic rather than static check)
- ▶ Bigger change to how code generation works.



Background

Full laziness

- ▶ Full laziness was introduced in John Hughes' 1983 PhD thesis *The design and implementation of programming languages*.

```
let f x = g x `op` {- .. expensive .. -}
```

Full laziness

- ▶ Full laziness was introduced in John Hughes' 1983 PhD thesis *The design and implementation of programming languages*.

```
let f x = g x `op` {- .. expensive .. -}
```

- ▶ Main reference for ghc is *Let-floating: moving bindings to give faster programs*, Simon Peyton Jones *et al.*, ICFP 1996:

Overall, full laziness buys a reduction of 10% in allocation, and 7% in instructions executed.

Full laziness

- ▶ Full laziness was introduced in John Hughes' 1983 PhD thesis *The design and implementation of programming languages*.

```
let f x = g x `op` {- .. expensive .. -}
```

- ▶ Main reference for ghc is *Let-floating: moving bindings to give faster programs*, Simon Peyton Jones et al., ICFP 1996:

Overall, full laziness buys a reduction of 10% in allocation, and 7% in instructions executed.

one program (..) is dramatically improved by full laziness. Leaving bindings stuck inside an inner loop is a Really Bad Idea

Full laziness

- ▶ Full laziness was introduced in John Hughes' 1983 PhD thesis *The design and implementation of programming languages*.

```
let f x = g x `op` {- .. expensive .. -}
```

- ▶ Main reference for ghc is *Let-floating: moving bindings to give faster programs*, Simon Peyton Jones et al., ICFP 1996:

Overall, full laziness buys a reduction of 10% in allocation, and 7% in instructions executed.

one program (..) is dramatically improved by full laziness. Leaving bindings stuck inside an inner loop is a Really Bad Idea

- ▶ It was already aware of the problem:

There is a final disadvantage to full laziness which is much more slippery: it may cause a space leak. (..) As our measurements show, space leaks do not seem to be a problem for real programs.

State hack

Increases the arity of functions:

```
foo :: Int -> IO ()  
foo x = do  
    let y = ...  
    ..
```

is really

```
foo :: Int -> State# RealWorld -> ..  
foo x = let y = .. in \w -> ..
```

which should be optimized to

```
foo :: Int -> State# RealWorld -> ..  
foo x w = let y = .. in ..
```

to increase the arity of foo.

Current benchmark estimates are that it improves performance by 5-15%.

- ▶ Perhaps it would be more natural to add call by name functions

$$a \xrightarrow{cbn} b$$

Nonetheless, I would prefer to say, once and for all, that (say) conduits should *never* be shared, no matter which function we apply.

²<https://gitlab.haskell.org/ghc/ghc/-/issues/18202>, Sylvain Henry

Call-by-name *functions*

- ▶ Perhaps it would be more natural to add call by name functions

$$a \xrightarrow{cbn} b$$

Nonetheless, I would prefer to say, once and for all, that (say) conduits should *never* be shared, no matter which function we apply.

- ▶ In a way, ghc already has functions of type²

$$a \xrightarrow{\text{oneshot}} b$$

but it's quite indirect (it enables floating in of `let` bindings; it's a generalization of the state hack).

²<https://gitlab.haskell.org/ghc/ghc/-/issues/18202>, Sylvain Henry